Algorithms (CS302) Project – Part 1 Divide and Conquer	**GROUP MEMBERS** Sana Munir Alam [24K-0573] Adeena Asif [24K-0628] Sharjeel Memon [24K-0555] **Program / Section:** BCS-5E

Part 1: Divide and Conquer

Pseudocode, analysis, code and sample output

Part (a): Largest and smallest element (Levitin 5.1, Exercise 2)

Algorithm in words

1. If the subarray has 1 element, that element is both the minimum and the maximum (0 comparisons).
2. If it has 2 elements, compare them once to decide which is the minimum and which is the maximum (1 comparison).
3. Otherwise, split the array into a left half and a right half and solve each recursively. Each call returns a pair (min, max).
4. Combine: the overall minimum is the smaller of the two minimums (1 comparison). The overall maximum is the larger of the two maximums (1 comparison).

Pseudocode

```
ALGORITHM MinMax(A[l..r])
 // Output: pair (minval, maxval) of A[l..r]
 if l = r then return (A[l], A[l])
 if r - l = 1 then
    if A[l] < A[r] then return (A[l], A[r])
    else return (A[r], A[l])
 mid <- floor((l + r) / 2)
 (min1, max1) <- MinMax(A[l..mid])
 (min2, max2) <- MinMax(A[mid+1..r])
 return ( min(min1, min2), max(max1, max2) )
```

Algorithms (CS302)
Project – Part 1
Divide and Conquer

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

Recurrence relation

The two recursive calls cost C(n/2) each. The combine step costs 2 comparisons. A subarray of size 2 costs 1 comparison.

$$C(n) = 2C\left(\frac{n}{2}\right) + 2 \;;\; \text{for n} > 2,$$

$$C(2) = 1$$

Solution

Solving by repeated substitution:

$$C(n) = 2C\left(\frac{n}{2}\right) + 2$$

$$= 2\left[2C\left(\frac{n}{4}\right) + 2\right] + 2 = 4C\left(\frac{n}{4}\right) + 4 + 2$$

$$= 4\left[2C\left(\frac{n}{8}\right) + 2\right] + 4 + 2 = 8C\left(\frac{n}{8}\right) + 8 + 4 + 2$$

$$= 8\left[2C\left(\frac{n}{16}\right) + 2\right] + 8 + 4 + 2 = 16C\left(\frac{n}{16}\right) + 16 + 8 + 4 + 2$$

After k steps the pattern is

$$C(n) = 2^k C\left(\frac{n}{2^k}\right) + \sum_{i=1}^{k} 2^i = 2^k C\left(\frac{n}{2^k}\right) + 2(2^k - 1)$$

The base case is $C(2) = 1$, so the recursion stops when $n/2^k = 2$:

$$\frac{n}{2^k} = 2 \quad \Rightarrow \quad 2^k = \frac{n}{2} \quad \Rightarrow \quad k = \log_2 n - 1$$

Substituting back in:

$$C(n) = \frac{n}{2} \cdot C(2) + 2\left(\frac{n}{2} - 1\right) = \frac{n}{2} + n - 2 = \frac{3n}{2} - 2$$

So $C(n) = \frac{3n}{2} - 2 \in \Theta(n)$. Master theorem check: $a = 2, b = 2, f(n) = 2 = \Theta(1) = O(n^{\log_2 2 - \varepsilon})$, so case 1 applies and $C(n) = \Theta(n)$.

Answer: C(n) = 3n/2 – 2, which is in Θ(n).

Algorithms (CS302) Project – Part 1 Divide and Conquer	GROUP MEMBERS Sana Munir Alam [24K-0573] Adeena Asif [24K-0628] Sharjeel Memon [24K-0555]
	Program / Section: BCS-5E

Comparison with brute force

Brute force: set min = max = A[0]. For each of the other n − 1 elements, compare with the current min and with the current max.

C_brute(n) = 2(n - 1) = 2n - 2

	Divide and conquer	Brute force
Comparisons	3n/2 − 2	2n − 2
Efficiency class	Θ(n)	Θ(n)
Ratio for large n	3/4	1

Both are Θ(n), but divide and conquer makes about **25% fewer comparisons** (ratio → 3/4). The saving does not come from the recursion itself. It comes from comparing elements in pairs first, so that each pair is used for both the min and the max.

Code

```
#include <iostream>
#include <vector>
#include <utility>
#include <algorithm>
using namespace std;

long long cmpCount = 0;

pair<int, int> minMax(const vector<int>& A, int l, int r) {
  if (l == r) return {A[l], A[l]};

  if (r - l == 1) {
    cmpCount++;
    return A[l] < A[r] ? make_pair(A[l], A[r]) : make_pair(A[r], A[l]);
  }

  int m = (l + r) / 2;
  auto L = minMax(A, l, m);
  auto R = minMax(A, m + 1, r);

  cmpCount += 2;
  return {min(L.first, R.first), max(L.second, R.second)};
```

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

```
}

int main() {
   int n;
   cout << "Enter n (>= 10): ";
   cin >> n;

   if (n < 10) {
      cout << "n must be >= 10\n";
      return 1;
   }

   vector<int> A(n);
   cout << "Enter " << n << " numbers: ";

   for (int &x : A) cin >> x;

   auto res = minMax(A, 0, n - 1);

   cout << "Min = " << res.first << ", Max = " << res.second << "\n";
   cout << "Comparisons (divide & conquer) = " << cmpCount << "\n";
   cout << "Comparisons (brute force)     = " << 2LL * (n - 1) << "\n";

   return 0;
}
```

Output

```
1 COde/Q1_Part_A.cpp/ Q1_Part_A
sanamuniralam@Sanas-MacBook-Air DAA Project Part 1 COde % cd "/Users/sanamuniralam/Desktop/
_Part_A && "/Users/sanamuniralam/Desktop/DAA Project Part 1 COde/Q1_Part_A.cpp/"Q1_Part_A
Enter n (>= 10): 16
Enter 16 numbers: 1 8 7 12 13 16 2 4 5 9 21 8 8 11 81 11
Min = 1, Max = 81
Comparisons (divide & conquer) = 22
Comparisons (brute force)      = 30
sanamuniralam@Sanas-MacBook-Air Q1_Part_A.cpp %
```

Algorithms (CS302) Project – Part 1 Divide and Conquer	**GROUP MEMBERS** Sana Munir Alam [24K-0573] Adeena Asif [24K-0628] Sharjeel Memon [24K-0555] **Program / Section:** BCS-5E

Part (b): Exponentiation a^n (Levitin 5.1, Exercise 3)

Algorithm in words

The idea is to compute half of the exponent, then square it.

a^n = (a^(n/2))^2 if n is even
a^n = (a^((n-1)/2))^2 * a if n is odd
a^1 = a (base case)

In code: compute t = a^⌊n/2⌋ recursively, square it, and multiply by one extra a if n is odd.

Pseudocode

```
ALGORITHM Power(a, n)        // n >= 1
 if n = 1 then return a
 t <- Power(a, floor(n / 2))
 if n is even then return t * t       // 1 multiplication
 else return t * t * a             // 2 multiplications
```

Recurrence relation

To get a^n we make two recursive calls (one per half of the exponent) and then do one multiplication to combine them, so the recurrence relation is

$$M(n) = M(\lfloor n/2 \rfloor) + M(\lceil n/2 \rceil) + 1, \qquad M(1) = 0$$

Solution

For $n = 2^k$ this becomes $M(n) = 2M(n/2) + 1$ with $M(1) = 0$. Solving by repeated substitution:

$$M(n) = 2M\left(\frac{n}{2}\right) + 1$$

$$= 2\left[2M\left(\frac{n}{4}\right) + 1\right] + 1 = 4M\left(\frac{n}{4}\right) + 2 + 1$$

$$= 4\left[2M\left(\frac{n}{8}\right) + 1\right] + 2 + 1 = 8M\left(\frac{n}{8}\right) + 4 + 2 + 1$$

$$= 8\left[2M\left(\frac{n}{16}\right) + 1\right] + 4 + 2 + 1 = 16M\left(\frac{n}{16}\right) + 8 + 4 + 2 + 1$$

After k steps the pattern is

$$M(n) = 2^k M\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} 2^i = 2^k M\left(\frac{n}{2^k}\right) + 2^k - 1$$

The recursion stops when $n/2^k = 1$:

$$\frac{n}{2^k} = 1 \quad \Rightarrow \quad 2^k = n \quad \Rightarrow \quad k = \log_2 n$$

Substituting back in:

$$M(n) = n \cdot M(1) + n - 1 = n \cdot 0 + n - 1 = n - 1 = \Theta(n)$$

Master theorem check: $a = 2, b = 2, f(n) = 1 = \Theta(1) = O(n^{\log_2 2 - \varepsilon})$, so case 1 applies and $M(n) = \Theta(n)$.

Brute force multiplies a by itself $n - 1$ times, so the count is exactly the same and this split gives no gain. The problem is that both halves are almost the same subproblem and we compute it twice. If we compute the half once and square it, it gets much better:

```
FastPower(a, n)
    if n = 1: return a
    h ← FastPower(a, floor(n/2))                    // computed only once
    if n is even: return h × h
    else:         return h × h × a
```

For this version there is only one recursive call, on half of the exponent, plus one squaring (or two multiplications when n is odd). Taking the even case, the recurrence relation is $M(n) = M(n/2) + 1$ with $M(1) = 0$. Solving it the same way:

$$M(n) = M\left(\frac{n}{2}\right) + 1$$
$$= \left[M\left(\frac{n}{4}\right) + 1\right] + 1 = M\left(\frac{n}{4}\right) + 2$$
$$= \left[M\left(\frac{n}{8}\right) + 1\right] + 2 = M\left(\frac{n}{8}\right) + 3$$
$$= \left[M\left(\frac{n}{16}\right) + 1\right] + 3 = M\left(\frac{n}{16}\right) + 4$$

After k steps the pattern is

$$M(n) = M\left(\frac{n}{2^k}\right) + k$$

The recursion stops when $n/2^k = 1$:

$$\frac{n}{2^k} = 1 \quad \Rightarrow \quad 2^k = n \quad \Rightarrow \quad k = \log_2 n$$

Substituting $k = \log_2 n$ back in:

$$\boldsymbol{M(n) = M(1) + \log_2 n = 0 + \log_2 n = \Theta(\log n)}$$

Master theorem check: $a = 1, b = 2, f(n) = 1 = \Theta(1) = \Theta(n^{\log_2 1})$, so case 2 applies and $M(n) = \Theta(\log n)$.

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

Comparison with brute force

Brute force: result = a, then multiply by a another n − 1 times. So the cost is n − 1 multiplications, which is Θ(n).

	One-call halving	Brute force
Multiplications	between log n and 2 log n	n − 1
Efficiency class	Θ(log n)	Θ(n)
Example n = 20	5	19

The halving algorithm is exponentially faster. Brute force needs about n multiplications; halving needs about log n.

Code

```
#include <iostream>
using namespace std;

long long mults = 0;

long long power(long long a, int n) {
  if (n == 1) return a;
  long long t = power(a, n / 2);
  mults++;
  long long sq = t * t;
  if (n % 2 == 0) return sq;
  mults++;
  return sq * a;
}

int main() {
  long long a;
  int n;
  cout << "Enter base a and exponent n (n >= 10): ";
  cin >> a >> n;

  if (n < 10) {
    cout << "n must be >= 10\n";
    return 1;
```

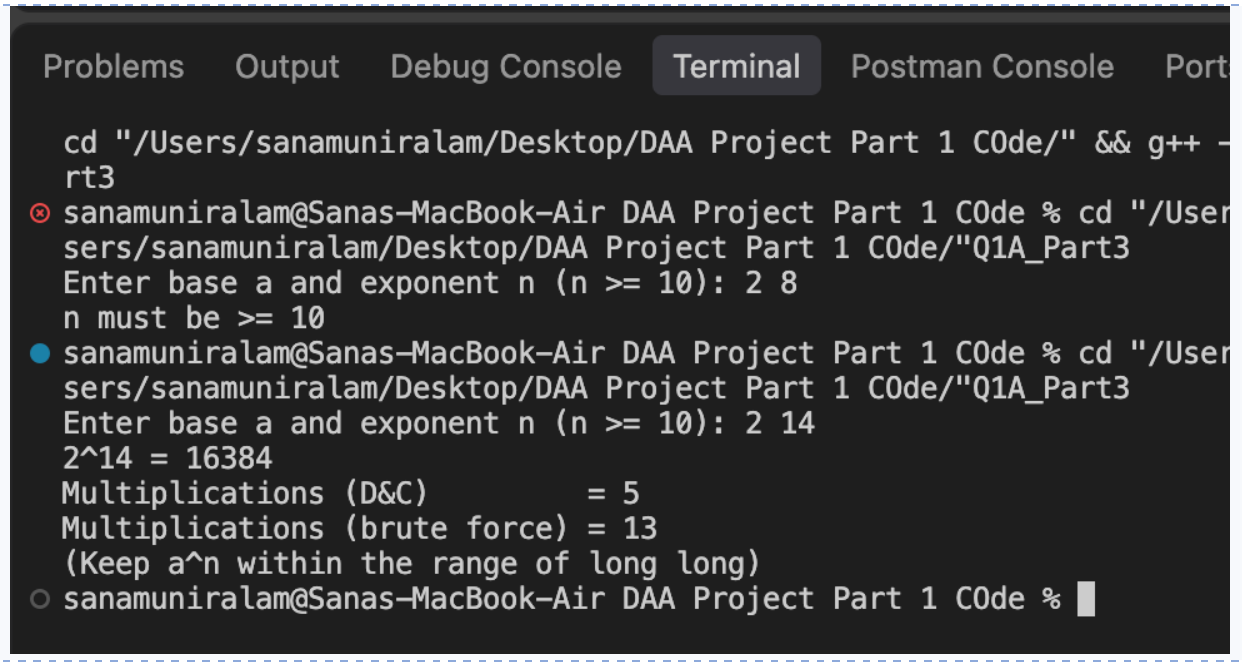
GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

```
  }

  long long result = power(a, n);
  cout << a << "^" << n << " = " << result << "\n";
  cout << "Multiplications (D&C)       = " << mults << "\n";
  cout << "Multiplications (brute force) = " << n - 1 << "\n";
  cout << "(Keep a^n within the range of long long)\n";
  return 0;
}
```

Output



Algorithms (CS302) Project – Part 1 Divide and Conquer	GROUP MEMBERS Sana Munir Alam [24K-0573] Adeena Asif [24K-0628] Sharjeel Memon [24K-0555] **Program / Section:** BCS-5E

Part (c): Counting inversions (Levitin 5.1, Exercise 9)

Definition: a pair (A[i], A[j]) is an inversion if i < j but A[i] > A[j].

Algorithm in words (modified merge sort)

1. Split the array into a left half and a right half.
2. Recursively sort each half and count its inversions.
3. Merge the two sorted halves. Whenever the element taken from the **right** half is smaller than the current element of the **left** half, it is smaller than **all** remaining left elements (the left half is sorted). That gives (mid − i + 1) new inversions, all crossing the split.
4. Total = inversions in left + inversions in right + crossing inversions.

Pseudocode

```
ALGORITHM SortCount(A[l..r])
 if l >= r then return 0
 mid <- floor((l + r) / 2)
 count <- SortCount(A[l..mid]) + SortCount(A[mid+1..r])
 i <- l;  j <- mid + 1;  k <- l
 while i <= mid and j <= r do
   if A[i] <= A[j] then  T[k++] <- A[i++]
   else  T[k++] <- A[j++];  count <- count + (mid - i + 1)
 copy the rest of the left half, then of the right half, into T
 copy T[l..r] back into A[l..r]
 return count
```

Recurrence relation

The algorithm makes two recursive calls on halves of size $n/2$, and the merge goes through each element once, which costs cn for some constant c. A single element takes constant time, so

$$T(n) = 2T\left(\frac{n}{2}\right) + cn, \qquad T(1) = c$$

Solution

Solving by repeated substitution:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

$$= 2\left[2T\left(\frac{n}{4}\right) + c\frac{n}{2}\right] + cn = 4T\left(\frac{n}{4}\right) + 2cn$$

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

$$= 4\left[2T\left(\frac{n}{8}\right) + c\frac{n}{4}\right] + 2cn = 8T\left(\frac{n}{8}\right) + 3cn$$
$$= 8\left[2T\left(\frac{n}{16}\right) + c\frac{n}{8}\right] + 3cn = 16T\left(\frac{n}{16}\right) + 4cn$$

After k steps the pattern is

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kcn$$

The recursion stops when the subproblem has size 1, i.e. when $n/2^k = 1$:

$$\frac{n}{2^k} = 1 \quad \Rightarrow \quad 2^k = n \quad \Rightarrow \quad k = \log_2 n$$

Substituting $k = \log_2 n$ back in:

$$\boldsymbol{T(n) = n \cdot c + cn\log_2 n = cn + cn\log_2 n = \Theta(n\log n)}$$

Master theorem check: $a = 2, b = 2, f(n) = cn = \Theta(n) = \Theta(n^{\log_2 2})$, so case 2 applies and $T(n) = \Theta(n\log n)$.

Checking every pair would be $\Theta(n^2)$.

Trace on A = [2, 4, 1, 3, 5]

Left half [2,4]: 0 inversions. Right half [1,3,5]: 0 inversions.
Merge [2,4] with [1,3,5]:
compare 2 and 1 -> 2 > 1, take 1, add (mid - i + 1) = 2 (pairs (2,1),(4,1))
compare 2 and 3 -> take 2
compare 4 and 3 -> 4 > 3, take 3, add 1 (pair (4,3))
compare 4 and 5 -> take 4, then 5
Total = 0 + 0 + 3 = 3.

Complexity

T(n) = 2 T(n/2) + Theta(n), T(1) = 0 => T(n) in Theta(n log n)

The merge step is linear and the counting adds only O(1) per element taken from the right half. Space is Θ(n) for the temporary array.

Common mistake: using < instead of <= in the merge. Equal elements are not inversions, so on ties the left element must be taken first.

Algorithms (CS302) Project – Part 1 Divide and Conquer	**GROUP MEMBERS** **Sana Munir Alam [24K-0573]** **Adeena Asif [24K-0628]** **Sharjeel Memon [24K-0555]** **Program / Section:** BCS-5E

Code

```
#include <iostream>
#include <vector>
using namespace std;

long long mergeCount(vector<int>& A, vector<int>& T, int l, int m, int r) {
   long long inv = 0;
   int i = l, j = m + 1, k = l;
   while (i <= m && j <= r) {
      if (A[i] <= A[j]) {
         T[k++] = A[i++];
      } else {
         T[k++] = A[j++];
         inv += m - i + 1;
      }
   }

   while (i <= m) T[k++] = A[i++];
   while (j <= r) T[k++] = A[j++];
   for (i = l; i <= r; i++) A[i] = T[i];
   return inv;
}

long long sortCount(vector<int>& A, vector<int>& T, int l, int r) {
   if (l >= r) return 0;
   int m = l + (r - l) / 2;
   long long leftInv = sortCount(A, T, l, m);
   long long rightInv = sortCount(A, T, m + 1, r);
   long long crossInv = mergeCount(A, T, l, m, r);
   return leftInv + rightInv + crossInv;
}

int main() {
   int n;
   cout << "Enter n (>= 10): ";
   cin >> n;

   if (n < 10) {
```

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

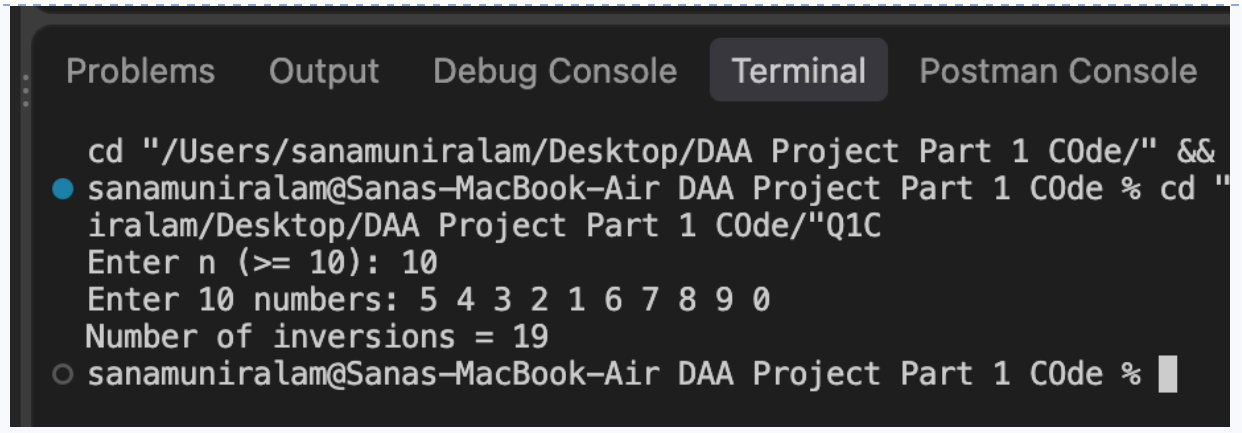
Program / Section: BCS-5E

```
    cout << "n must be >= 10\n";
    return 1;
  }

  vector<int> A(n), T(n);

  cout << "Enter " << n << " numbers: ";
  for (int& x : A) cin >> x;
  cout << "Number of inversions = " << sortCount(A, T, 0, n - 1) << "\n";
  return 0;
}
```

Output



Algorithms (CS302)
Project – Part 1
Divide and Conquer

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

Part (d): Quicksort (Levitin, Section 5.2)

Quicksort picks a pivot, partitions the array so that smaller elements are on the left and larger ones on the right, and then sorts both sides recursively. The partition step costs about n comparisons. The total cost depends on how balanced the split is.

Recurrence relations

Partitioning n elements costs about n comparisons. In the best case the pivot splits the array into two halves, and in the worst case it leaves one part with $n - 1$ elements.

Best case: the pivot always lands in the middle. Both subarrays have size about n/2.

```
C_best(n) = 2 C_best(n/2) + n,   C_best(1) = 0
```

By the Master theorem (a = 2, b = 2, d = 1, a = b^d) this gives

```
C_best(n) = n log n   =>   Theta(n log n)
```

Worst case: the pivot is always the smallest or largest element. This happens, for example, on an already sorted array when the first element is the pivot. One subarray is empty and the other has n − 1 elements. With Levitin's partition (about n + 1 comparisons for n elements):

```
C_worst(n) = C_worst(n-1) + (n+1)
           = (n+1) + n + (n-1) + ... + 3
           = (n+1)(n+2)/2 - 3   =>   Theta(n^2)
```

Case	When it happens	Comparisons	Class
Best	Pivot always splits evenly	n log n	Θ(n log n)
Average	Random input	about 1.39 n log n (2n ln n)	Θ(n log n)
Worst	Pivot always the min or max (sorted input with first-element pivot)	(n+1)(n+2)/2 − 3	Θ(n^2)

To avoid the worst case, choose a random pivot or the median of three elements.

Solution

Best case: $C(n) = 2C(n/2) + n$ with $C(1) = 0$. Solving by repeated substitution:

$$C(n) = 2C\left(\frac{n}{2}\right) + n$$

$$= 2\left[2C\left(\frac{n}{4}\right) + \frac{n}{2}\right] + n = 4C\left(\frac{n}{4}\right) + 2n$$

$$= 4\left[2C\left(\frac{n}{8}\right) + \frac{n}{4}\right] + 2n = 8C\left(\frac{n}{8}\right) + 3n$$

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

$$= 8\left[2C\left(\frac{n}{16}\right) + \frac{n}{8}\right] + 3n = 16C\left(\frac{n}{16}\right) + 4n$$

After k steps the pattern is

$$C(n) = 2^k C\left(\frac{n}{2^k}\right) + kn$$

The recursion stops when the subproblem has size 1, i.e. when $n/2^k = 1$:

$$\frac{n}{2^k} = 1 \quad \Rightarrow \quad 2^k = n \quad \Rightarrow \quad k = \log_2 n$$

Substituting $k = \log_2 n$ back in:

$$\boldsymbol{C(n) = n \cdot 0 + n\log_2 n = n\log_2 n = \Theta(n\log n)}$$

Master theorem check: $a = 2, b = 2, f(n) = n = \Theta(n) = \Theta(n^{\log_2 2})$, so case 2 applies and $C(n) = \Theta(n\log n)$.

Worst case: $C(n) = C(n-1) + n + 1$ with $C(1) = 0$. The problem only shrinks by one element here, so the Master theorem does not apply and we use repeated substitution:

$$\begin{aligned} C(n) &= C(n-1) + (n+1) \\ &= C(n-2) + n + (n+1) \\ &= C(n-3) + (n-1) + n + (n+1) \\ &= C(n-4) + (n-2) + (n-1) + n + (n+1) \end{aligned}$$

After k steps the pattern is

$$C(n) = C(n-k) + \sum_{i=0}^{k-1} (n+1-i)$$

The recursion stops when $n - k = 1$:

$$n - k = 1 \quad \Rightarrow \quad k = n - 1$$

Substituting back in:

$$\boldsymbol{C(n) = C(1) + \sum_{i=0}^{n-2} (n+1-i) = 0 + (n+1) + n + \cdots + 3 = \frac{(n+1)(n+2)}{2} - 3 = \Theta(n^2)}$$

Algorithms (CS302) Project – Part 1 Divide and Conquer	GROUP MEMBERS Sana Munir Alam [24K-0573] Adeena Asif [24K-0628] Sharjeel Memon [24K-0555]
	Program / Section: BCS-5E

Code

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cfloat>
#include <climits>
using namespace std;

double closest(vector<double>& a, int l, int r) {
  if (l >= r) return DBL_MAX;
  int m = (l + r) / 2;
  nth_element(a.begin() + l, a.begin() + m, a.begin() + r + 1);

  double d1 = closest(a, l, m);
  double d2 = closest(a, m + 1, r);
  double minS2 = *min_element(a.begin() + m + 1, a.begin() + r + 1);
  double maxS1 = a[m];

  return min({d1, d2, minS2 - maxS1});
}

int main() {
  int n;
  cout << "Enter n (>= 10): "; cin >> n;

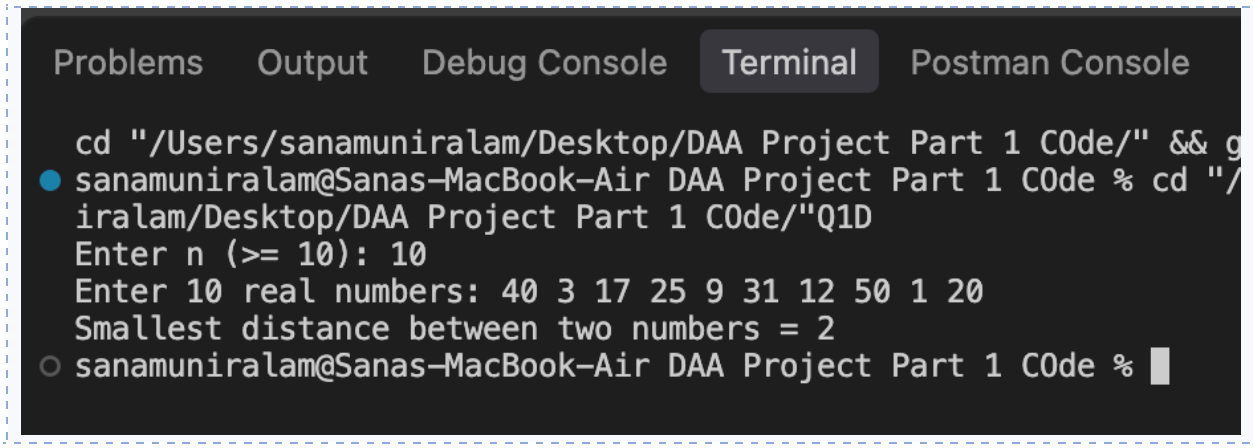
  if (n < 10) { cout << "n must be >= 10\n"; return 1; }
  vector<double> a(n);
  cout << "Enter " << n << " real numbers: ";

  for (double &x : a) cin >> x;
  cout << "Smallest distance between two numbers = " << closest(a, 0, n - 1) << "\n";
  return 0;
}
```

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

Output



Part (e): 1-D closest pair by divide and conquer (Levitin 5.5, Exercise 1)

Problem: to find the two closest numbers in a set of n real numbers using divide and conquer, give the efficiency class, and say whether it is a good algorithm.

Algorithm in word

1. If there are fewer than 2 numbers, there is no pair (return ∞).
2. Find the **median** and split the set into S_1 (the smaller half) and S_2 (the larger half). Every element of S_1 is ≤ every element of S_2.
3. Recursively find d_1 = closest distance inside S_1 and d_2 = closest distance inside S_2.
4. A closest pair is either inside S_1, inside S_2, or has one number in each. Because S_1 lies entirely to the left of S_2, the best crossing pair is **(max of S_1, min of S_2)**. So the crossing distance is $\min(S_2) - \max(S_1)$.
5. Return $\min(d_1, d_2, \min(S_2) - \max(S_1))$.

Pseudocode

```
ALGORITHM ClosestPair1D(A, l, r)
  if l >= r then
    return ∞

  mid ← floor((l + r) / 2)

  d1 ← ClosestPair1D(A, l, mid)
  d2 ← ClosestPair1D(A, mid + 1, r)
```

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

```
maxLeft ← maximum element in A[l..mid]
minRight ← minimum element in A[mid+1..r]

d3 ← minRight - maxLeft

return min(d1, d2, d3)
```

Recurrence relation

Each of the two recursive calls works on half of the numbers, and finding the median and the min and max is linear work, cn for some constant c. So

$$T(n) = 2T\left(\frac{n}{2}\right) + cn, \qquad T(1) = c$$

Solution

Solving by repeated substitution:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$
$$= 2\left[2T\left(\frac{n}{4}\right) + c\frac{n}{2}\right] + cn = 4T\left(\frac{n}{4}\right) + 2cn$$
$$= 4\left[2T\left(\frac{n}{8}\right) + c\frac{n}{4}\right] + 2cn = 8T\left(\frac{n}{8}\right) + 3cn$$
$$= 8\left[2T\left(\frac{n}{16}\right) + c\frac{n}{8}\right] + 3cn = 16T\left(\frac{n}{16}\right) + 4cn$$

After k steps the pattern is

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kcn$$

The recursion stops when the subproblem has size 1, i.e. when $n/2^k = 1$:

$$\frac{n}{2^k} = 1 \quad \Rightarrow \quad 2^k = n \quad \Rightarrow \quad k = \log_2 n$$

Substituting $k = \log_2 n$ back in:

$$\boldsymbol{T(n) = n \cdot c + cn\log_2 n = cn + cn\log_2 n = \Theta(n\log n)}$$

Master theorem check: $a = 2$, $b = 2$, $f(n) = cn = \Theta(n) = \Theta(n^{\log_2 2})$, so case 2 applies and

$$\boldsymbol{T(n) = \Theta(n\log n).}$$

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

Is it a good algorithm?

It achieves Θ(n log n)time, but sorting the numbers and scanning adjacent elements is simpler and achieves the same efficiency class.

Warning about a common wrong answer: if you sort first and then run a recursive search on the sorted array, the recursion is pointless, because the sorting already dominates and a single scan is enough.

Code

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
#include <limits>
using namespace std;

double closest_1d(vector<double> arr)
{
  int n = arr.size();

  if (n < 2)
    return numeric_limits<double>::infinity();

  if (n == 2)
    return abs(arr[0] - arr[1]);

  vector<double> sorted = arr;
  sort(sorted.begin(), sorted.end());

  double m = sorted[n / 2];  // Median

  vector<double> left, right;

  for (double x : arr)
  {
    if (x < m)
      left.push_back(x);
    else
      right.push_back(x);
```

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

```
    }

    if (left.empty())
        return 0;

    double d = min(closest_1d(left), closest_1d(right));

    double cross = *min_element(right.begin(), right.end())
                 - *max_element(left.begin(), left.end());

    return min(d, cross);
}

void run_e()
{
    int n;

    cout << "Enter number of elements (minimum 10): ";
    cin >> n;

    while (n < 10)
    {
        cout << "Please enter at least 10 elements: ";
        cin >> n;
    }

    vector<double> a(n);

    cout << "Enter " << n << " array elements: ";

    for (int i = 0; i < n; i++)
        cin >> a[i];

    cout << "\nArray: ";

    for (double x : a)
        cout << x << " ";

    cout << endl;
```

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

```
  vector<double> s = a;
  sort(s.begin(), s.end());

  double check = numeric_limits<double>::infinity();

  for (int i = 0; i < n - 1; i++)
    check = min(check, s[i + 1] - s[i]);

  cout << "Smallest distance (divide & conquer) = "
     << closest_1d(a)
     << " | sort-and-scan check = " << check << endl;

  cout << "Efficiency: T(n) = 2T(n/2) + O(n) -> "
     << "Theta(n log n). Sorting + scanning is also "
     << "Theta(n log n) and simpler." << endl;
}

int main()
{
  run_e();
  return 0;
}
```

Output

```
Enter number of elements (minimum 10): 20
Enter 20 array elements: 12 5 8 21 3 17 9 14 6 25 11 2 19 7 15 4 23 10 18 13

Array: 12 5 8 21 3 17 9 14 6 25 11 2 19 7 15 4 23 10 18 13
Smallest distance (divide & conquer) = 1 | sort-and-scan check = 1
Efficiency: T(n) = 2T(n/2) + O(n) -> Theta(n log n). Sorting + scanning is also Theta(n log n) and simpler.

--------------------------------
Process exited after 39.86 seconds with return value 0
Press any key to continue . . .
```

Algorithms (CS302) Project – Part 1 Divide and Conquer	GROUP MEMBERS Sana Munir Alam [24K-0573] Adeena Asif [24K-0628] Sharjeel Memon [24K-0555] **Program / Section:** BCS-5E

Part (f): Kleinberg, Solved Exercise 1: finding the peak of a unimodal array

Problem: array A[1..n] of distinct numbers that increase up to some position p and then decrease. Find p while reading only O(log n) entries.

1.Idea: do constant work, throw away half

To get O(log n), do a constant amount of work, discard half of the input, and recurse on what is left (exactly like binary search). Reading only A[mid] is not enough, because we cannot tell if mid is on the rising side or the falling side. So we also read its two neighbours.

Algorithm in words

Let mid be the middle index of the current range. Read A[mid − 1], A[mid], A[mid + 1]. There are three cases:

Case	Meaning	Action
A[mid − 1] < A[mid] < A[mid + 1]	mid is on the **up-slope**, so p is after mid	continue on mid + 1 … hi
A[mid − 1] > A[mid] > A[mid + 1]	mid is on the **down-slope**, so p is before mid	continue on lo … mid − 1
A[mid] > both neighbours	mid is the peak	return mid

At the ends of the range, treat a missing neighbour as −∞.

Pseudocode

```
ALGORITHM FindPeak(A[1..n])
 lo <- 1;  hi <- n
 while lo <= hi do
    mid <- floor((lo + hi) / 2)
    if A[mid-1] < A[mid] and A[mid] > A[mid+1] then return mid
    else if A[mid-1] < A[mid] < A[mid+1] then lo <- mid + 1
    else hi <- mid - 1
```

Recurrence relation

Each call reads at most 3 entries, which is constant work c, and then continues on only one half of the array, so

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

$$T(n) = T\left(\frac{n}{2}\right) + c, \qquad T(1) = c$$

Solution

Solving by repeated substitution:

$$T(n) = T\left(\frac{n}{2}\right) + c$$
$$= \left[T\left(\frac{n}{4}\right) + c\right] + c = T\left(\frac{n}{4}\right) + 2c$$
$$= \left[T\left(\frac{n}{8}\right) + c\right] + 2c = T\left(\frac{n}{8}\right) + 3c$$
$$= \left[T\left(\frac{n}{16}\right) + c\right] + 3c = T\left(\frac{n}{16}\right) + 4c$$

After k steps the pattern is

$$T(n) = T\left(\frac{n}{2^k}\right) + kc$$

The recursion stops when $n/2^k = 1$:

$$\frac{n}{2^k} = 1 \quad \Rightarrow \quad 2^k = n \quad \Rightarrow \quad k = \log_2 n$$

Substituting $k = \log_2 n$ back in:

$$\boldsymbol{T(n) = T(1) + c\log_2 n = c + c\log_2 n = \Theta(\log n)}$$

Master theorem check: $a = 1, b = 2, f(n) = c = \Theta(1) = \Theta(n^{\log_2 1})$, so case 2 applies and $T(n) = \Theta(\log n)$.

General remark from the book: the same O(log n) bound holds whenever each step throws away any constant fraction of the input.

Code

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

int main() {
  int n;
  cout << "Enter n (>= 10): "; cin >> n;

  if (n < 10) { cout << "n must be >= 10\n"; return 1; }
```

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

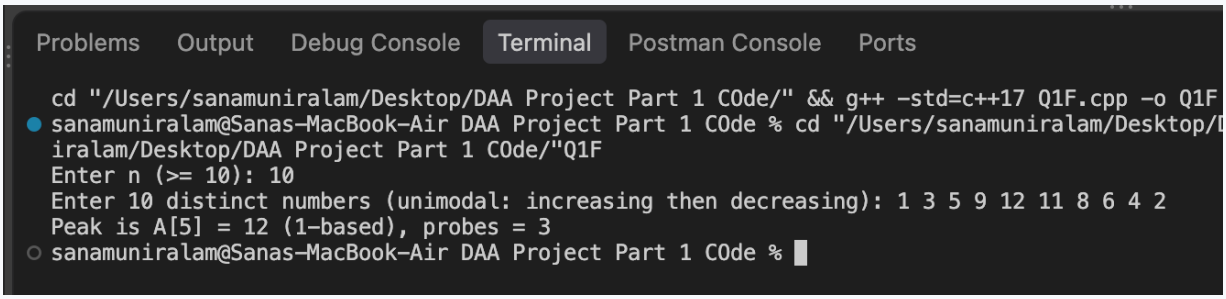
```
  vector<int> A(n);
  cout << "Enter " << n << " distinct numbers (unimodal: increasing then decreasing): ";
  for (int &x : A) cin >> x;

  int lo = 0, hi = n - 1, probes = 0, peak = -1;
  while (lo <= hi) {
    int m = lo + (hi - lo) / 2;
    int cur = A[m]; probes++;
    int left = (m - 1 >= lo) ? (probes++, A[m - 1]) : INT_MIN;
    int right = (m + 1 <= hi) ? (probes++, A[m + 1]) : INT_MIN;

    if (cur > left && cur > right) { peak = m; break; }
    else if (left < cur && cur < right) lo = m + 1;
    else hi = m - 1;
  }

  cout << "Peak is A[" << peak + 1 << "] = " << A[peak] << " (1-based), probes = " << probes
<< "\n";
  return 0;
}
```

Output



Algorithms (CS302) Project – Part 1 Divide and Conquer	GROUP MEMBERS Sana Munir Alam [24K-0573] Adeena Asif [24K-0628] Sharjeel Memon [24K-0555] **Program / Section:** BCS-5E

Part (g): Kleinberg, Solved Exercise 2: best day to buy and sell a stock

Problem: prices p(1), …, p(n). Choose a buy day i and a later sell day j to maximise p(j) − p(i). Brute force tries all pairs in $O(n^2)$. Required: O(n log n).

Example from the book: n = 3, p = (9, 1, 5). Answer: buy on day 2, sell on day 3, profit 4 per share.

Algorithm in words

Split the days into S = first half and S' = second half. An optimal answer is one of three kinds:

1. Both days lie in S. Solve S recursively.
2. Both days lie in S'. Solve S' recursively.
3. The buy day i is in S and the sell day j is in S'. Then the profit p(j) − p(i) is largest when p(i) is the **minimum price in S** and p(j) is the **maximum price in S'**.

Return the best of the three.

Power-of-2 assumption (from the book): if n is not a power of 2, let n' be the next power of 2 and set p(i) = p(n) for n < i ≤ n'. The answer does not change and the input at most doubles.

Pseudocode

```
ALGORITHM BestTrade(p[l..r])
 if l = r then return (profit = -infinity, min-day = l, max-day = l)
 mid <- floor((l + r) / 2)
 Left  <- BestTrade(p[l..mid])
 Right <- BestTrade(p[mid+1..r])
 cross <- p[Right.maxDay] - p[Left.minDay]      // buy in left, sell in right
 best  <- the largest of Left.profit, Right.profit, cross  (remember the days)
 minDay <- day with smaller price of Left.minDay, Right.minDay
 maxDay <- day with larger price of Left.maxDay, Right.maxDay
 return (best, minDay, maxDay)
```

If the final best profit is ≤ 0, report that no money can be made.

Algorithms (CS302) Project – Part 1 Divide and Conquer	**GROUP MEMBERS** Sana Munir Alam [24K-0573] Adeena Asif [24K-0628] Sharjeel Memon [24K-0555] **Program / Section:** BCS-5E

Recurrence relation

Both halves are solved recursively, and then finding the cheapest day in the first half and the most expensive day in the second half is linear work, cn for some constant c. So

$$T(n) = 2T\left(\frac{n}{2}\right) + cn, \qquad T(1) = c$$

Solution

Solving by repeated substitution:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$
$$= 2\left[2T\left(\frac{n}{4}\right) + c\frac{n}{2}\right] + cn = 4T\left(\frac{n}{4}\right) + 2cn$$
$$= 4\left[2T\left(\frac{n}{8}\right) + c\frac{n}{4}\right] + 2cn = 8T\left(\frac{n}{8}\right) + 3cn$$
$$= 8\left[2T\left(\frac{n}{16}\right) + c\frac{n}{8}\right] + 3cn = 16T\left(\frac{n}{16}\right) + 4cn$$

After k steps the pattern is

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kcn$$

The recursion stops when the subproblem has size 1, i.e. when $n/2^k = 1$:

$$\frac{n}{2^k} = 1 \quad \Rightarrow \quad 2^k = n \quad \Rightarrow \quad k = \log_2 n$$

Substituting $k = \log_2 n$ back in:

$$\boldsymbol{T(n) = n \cdot c + cn\log_2 n = cn + cn\log_2 n = \Theta(n\log n)}$$

Master theorem check: $a = 2, b = 2, f(n) = cn = \Theta(n) = \Theta(n^{\log_2 2})$, so case 2 applies and $T(n) = \Theta(n\log n)$.

That is better than the $O(n^2)$ of trying every pair. If no trade makes a profit we report that. (The book mentions that dynamic programming can do it in $O(n)$.)

Code

```
#include <iostream>
#include <vector>
using namespace std;

struct Res {
    double best;
    int buy, sell, mn, mx;
};

Res solve(const vector<double>& p, int l, int r) {
    if (l == r) return {-1e18, -1, -1, l, l};
```

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

```
    int m = (l + r) / 2;

    Res L = solve(p, l, m), R = solve(p, m + 1, r);
    Res out;

    out.best = L.best; out.buy = L.buy; out.sell = L.sell;

    if (R.best > out.best) { out.best = R.best; out.buy = R.buy; out.sell = R.sell; }
    double cross = p[R.mx] - p[L.mn];

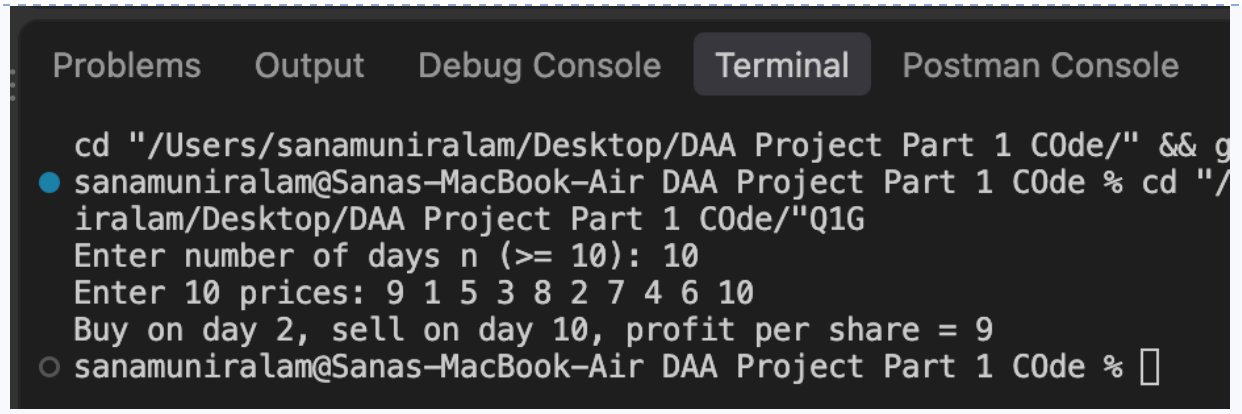
    if (cross > out.best) { out.best = cross; out.buy = L.mn; out.sell = R.mx; }
    out.mn = (p[L.mn] <= p[R.mn]) ? L.mn : R.mn;
    out.mx = (p[L.mx] >= p[R.mx]) ? L.mx : R.mx;
    return out;
}
int main() {
    int n;
    cout << "Enter number of days n (>= 10): "; cin >> n;
    if (n < 10) { cout << "n must be >= 10\n"; return 1; }

    vector<double> p(n);
    cout << "Enter " << n << " prices: ";

    for (double &x : p) cin >> x;
    Res r = solve(p, 0, n - 1);

    if (r.best <= 0) cout << "No way to make money in this period.\n";
    else cout << "Buy on day " << r.buy + 1 << ", sell on day " << r.sell + 1 << ", profit per share = "
<< r.best << "\n";
    return 0;
}
```

Output



Algorithms (CS302) Project – Part 1 Divide and Conquer	GROUP MEMBERS Sana Munir Alam [24K-0573] Adeena Asif [24K-0628] Sharjeel Memon [24K-0555] **Program / Section:** BCS-5E

Part (h), Exercise 2: Significant inversions

Problem: a significant inversion is a pair i < j with a_i > 2·a_j. Count them in O(n log n).

Idea: modify the inversion counter from Q1(c). The only change is that the crossing pairs are counted with the condition a_i > 2a_j **before** the normal merge.

Algorithm in words

1. Split the array into halves L and R. Recursively sort each half and count its significant inversions.
2. Now L and R are sorted. Keep a pointer p in L, starting at the first element.
3. For each x in R, from smallest to largest: while L[p] ≤ 2x, move p forward. Every element from L[p] to the end of L is greater than 2x, so add (|L| − p + 1) to the count.
4. The pointer never moves backward, because 2x grows as x grows. So this step is linear.
5. Do an ordinary merge so that the array is sorted for the level above.

Pseudocode

```
COUNT-SIGNIFICANT-INVERSIONS(A, n):
  if n <= 1: return 0
  Divide A into two halves L and R
  countLeft = COUNT-SIGNIFICANT-INVERSIONS(L, length(L))
  countRight = COUNT-SIGNIFICANT-INVERSIONS(R, length(R))
  countCross = COUNT-CROSS-INVERSIONS(L, R)
  MERGE L and R into A in sorted order
  return countLeft + countRight + countCross

COUNT-CROSS-INVERSIONS(L, R):
  count = 0; p = 1
  for j = 1 to length(R):
    while p <= length(L) AND L[p] <= 2 * R[j]:
      p = p + 1
    count = count + length(L) - p + 1
  return count

MERGE(L, R):
  i = 1; j = 1; k = 1
```

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

```
while i <= length(L) AND j <= length(R):
  if L[i] <= R[j]:
    A[k] = L[i]; i = i + 1
  else:
    A[k] = R[j]; j = j + 1
  k = k + 1
while i <= length(L):
  A[k] = L[i]; i = i + 1; k = k + 1
while j <= length(R):
  A[k] = R[j]; j = j + 1; k = k + 1
```

Recurrence relation

There are two recursive calls on halves, and then a linear counting pass and a linear merge, which together cost cn. So

$$T(n) = 2T\left(\frac{n}{2}\right) + cn, \qquad T(1) = c$$

Solution

Solving by repeated substitution:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$
$$= 2\left[2T\left(\frac{n}{4}\right) + c\frac{n}{2}\right] + cn = 4T\left(\frac{n}{4}\right) + 2cn$$
$$= 4\left[2T\left(\frac{n}{8}\right) + c\frac{n}{4}\right] + 2cn = 8T\left(\frac{n}{8}\right) + 3cn$$
$$= 8\left[2T\left(\frac{n}{16}\right) + c\frac{n}{8}\right] + 3cn = 16T\left(\frac{n}{16}\right) + 4cn$$

After k steps the pattern is

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kcn$$

The recursion stops when the subproblem has size 1, i.e. when $n/2^k = 1$:

$$\frac{n}{2^k} = 1 \quad \Rightarrow \quad 2^k = n \quad \Rightarrow \quad k = \log_2 n$$

Substituting $k = \log_2 n$ back in:

$$\boldsymbol{T(n) = n \cdot c + cn\log_2 n = cn + cn\log_2 n = \Theta(n\log n)}$$

Algorithms (CS302) Project – Part 1 Divide and Conquer	**GROUP MEMBERS** Sana Munir Alam [24K-0573] Adeena Asif [24K-0628] Sharjeel Memon [24K-0555] **Program / Section:** BCS-5E

Master theorem check: $a = 2, b = 2, f(n) = cn = \Theta(n) = \Theta(n^{\log_2 2})$, so case 2 applies and $T(n) = \Theta(n\log n)$.

Correctness

The recursive calls count significant inversions within (L) and (R), while COUNT-CROSS-INVERSIONS counts those between the halves using (L[p]>2R[j]). Since both halves are sorted, every significant inversion is counted exactly once. Merging then sorts the array for the next recursive call.

Trace on [6,2,1,5]

> Left [6,2]: sorted [2,6], pair (6,2): 6 > 2*2 -> 1 significant inversion
> Right [1,5]: sorted [1,5], pair (1,5): 1 > 10? no -> 0
> Cross with L = [2,6], R = [1,5]:
> x = 1: 2 <= 2 so p moves to 6; 6 <= 2? no. add 2 - 1 = 1 (pair (6,1))
> x = 5: 2 <= 10, 6 <= 10, p moves past the end. add 0
> Total = 1 + 0 + 1 = 2 (pairs (6,2) and (6,1))

Code

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

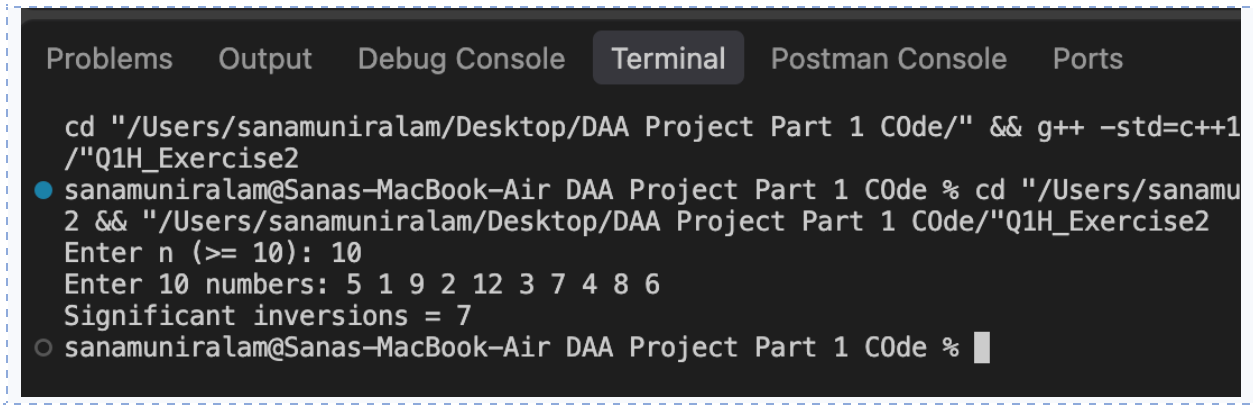
long long sortCount(vector<long long>& a, int l, int r) {
    if (l >= r) return 0;
    int m = (l + r) / 2;
    long long cnt = sortCount(a, l, m) + sortCount(a, m + 1, r);
    int p = l;
    for (int j = m + 1; j <= r; j++) {
        while (p <= m && a[p] <= 2 * a[j]) p++;
        cnt += m - p + 1;
    }
    inplace_merge(a.begin() + l, a.begin() + m + 1, a.begin() + r + 1);
    return cnt;
}

int main() {
    int n;
    cout << "Enter n (>= 10): "; cin >> n;
    if (n < 10) { cout << "n must be >= 10\n"; return 1; }
    vector<long long> a(n);
    cout << "Enter " << n << " numbers: ";
    for (auto &x : a) cin >> x;
    cout << "Significant inversions = " << sortCount(a, 0, n - 1) << "\n";
    return 0;
}
```

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

Output



Part (h), Exercise 3: More than n/2 equivalent bank cards

Problem: n cards. The only operation is the equivalence tester, which takes two cards and says whether they belong to the same account. Decide whether more than n/2 cards are all equivalent, using O(n log n) tester calls.

Key fact: if one account has more than n/2 of the n cards, then it has more than half of the cards in **at least one** of the two halves.

Proof: suppose it had at most half in the left half and at most half in the right half. Then its total would be at most n/2, which contradicts "more than n/2".

Algorithm in words

1. If there is one card, it is its own majority. Return it.
2. Split the cards into two halves. Recursively get a majority candidate from each half (or "none").
3. For each candidate, compare it with every card in the current range using the tester, and count the matches (the candidate counts itself without a test).
4. If a candidate's count is more than half of the range, return it. Otherwise return "none".
5. The answer to the bank's question is yes exactly when the top-level call returns a card.

Pseudocode

```
ALGORITHM Majority(cards[l..r])
 if l = r then return cards[l]
 mid <- floor((l + r) / 2)
```

Algorithms (CS302) Project – Part 1 Divide and Conquer	**GROUP MEMBERS** Sana Munir Alam [24K-0573] Adeena Asif [24K-0628] Sharjeel Memon [24K-0555]
	Program / Section: BCS-5E

```
for each candidate c in { Majority(cards[l..mid]), Majority(cards[mid+1..r]) }
  if c is not none then
    count <- number of cards in cards[l..r] equivalent to c   // uses the tester
    if count > (r - l + 1) / 2 then return c
return none
```

Recurrence relation

There are two recursive calls on halves, and then at most 2 candidates are compared with at most n cards each, which is at most $2n$ tests. A single card needs no test. So

$$T(n) = 2T\left(\frac{n}{2}\right) + 2n, \qquad T(1) = 0$$

Solution

Solving by repeated substitution:

$$T(n) = 2T\left(\frac{n}{2}\right) + 2n$$
$$= 2\left[2T\left(\frac{n}{4}\right) + 2 \cdot \frac{n}{2}\right] + 2n = 4T\left(\frac{n}{4}\right) + 4n$$
$$= 4\left[2T\left(\frac{n}{8}\right) + 2 \cdot \frac{n}{4}\right] + 4n = 8T\left(\frac{n}{8}\right) + 6n$$
$$= 8\left[2T\left(\frac{n}{16}\right) + 2 \cdot \frac{n}{8}\right] + 6n = 16T\left(\frac{n}{16}\right) + 8n$$

After k steps the pattern is

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + 2kn$$

The recursion stops when the subproblem has size 1, i.e. when $n/2^k = 1$:

$$\frac{n}{2^k} = 1 \quad \Rightarrow \quad 2^k = n \quad \Rightarrow \quad k = \log_2 n$$

Substituting $k = \log_2 n$ back in:

$$\boldsymbol{T(n) = n \cdot 0 + 2n\log_2 n = 2n\log_2 n = \Theta(n\log n)}$$

Master theorem check: $a = 2, b = 2, f(n) = 2n = \Theta(n) = \Theta(n^{\log_2 2})$, so case 2 applies and $T(n) = \Theta(n\log n)$.

Example

Take $n = 8$ cards whose hidden accounts are X, Y, X, X, Z, X, Y, X (cards 1 to 8). Account X has 5 of the 8 cards, so the answer should be yes.

Algorithms (CS302)
Project – Part 1
Divide and Conquer

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

```
[1-2] X,Y   candidates 1 and 2 both fail         2 tests -> none
[3-4] X,X   candidate 3 matches card 4           1 test  -> card 3
[1-4]       candidate 3 vs cards 1,2,4: y,n,y    3 tests -> card 3 (3 of 4)
[5-6] Z,X   both candidates fail                 2 tests -> none
[7-8] Y,X   both candidates fail                 2 tests -> none
[5-8]       no candidates                        0 tests -> none
[1-8]       candidate 3 vs the other 7 cards     7 tests -> 5 of 8 > 4
total: 17 tests, answer YES (account X)
```

Code

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> account;
long long tests = 0;

bool equivalent(int i, int j) { tests++; return account[i] == account[j]; }

int candidate(int l, int r) {
    if (l == r) return l;
    int m = (l + r) / 2;
    int cands[2] = {candidate(l, m), candidate(m + 1, r)};
    for (int c : cands) {
        if (c == -1) continue;
        int count = 0;
        for (int k = l; k <= r; k++)
            if (k == c || equivalent(c, k)) count++;
        if (2 * count > r - l + 1) return c;
    }
    return -1;
}

int main() {
    int n;
    cout << "Enter number of cards n (>= 10): "; cin >> n;
    if (n < 10) { cout << "n must be >= 10\n"; return 1; }

    account.resize(n);
    cout << "Enter an account number for each card: ";

    for (int &x : account) cin >> x;
    int c = candidate(0, n - 1);

    if (c == -1) cout << "No set of more than n/2 equivalent cards.\n";
    else cout << "YES: more than n/2 cards are equivalent to card " << c + 1 << "\n";
    cout << "Equivalence-tester calls = " << tests << "\n";
    return 0;
}
```

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

Output

```
Problems  Output  Debug Console  Terminal  Postman Console  Ports

cd "/Users/sanamuniralam/Desktop/DAA Project Part 1 COde/" && g++ -std=c++17 Q1
/"Q1H_Exercise3
sanamuniralam@Sanas-MacBook-Air DAA Project Part 1 COde % cd "/Users/sanamunira
3 && "/Users/sanamuniralam/Desktop/DAA Project Part 1 COde/"Q1H_Exercise3
Enter number of cards n (>= 10): 10
Enter an account number for each card: 1 1 1 2 1 3 1 4 1 1
YES: more than n/2 cards are equivalent to card 1
Equivalence-tester calls = 27
sanamuniralam@Sanas-MacBook-Air DAA Project Part 1 COde % cd "/Users/sanamunira
3 && "/Users/sanamuniralam/Desktop/DAA Project Part 1 COde/"Q1H_Exercise3
Enter number of cards n (>= 10): 10
Enter an account number for each card: 1 2 3 4 5 6 7 8 9 0
No set of more than n/2 equivalent cards.
Equivalence-tester calls = 12
sanamuniralam@Sanas-MacBook-Air DAA Project Part 1 COde %
```

Part (h), Exercise 6: Local minimum in a binary tree

Problem: complete binary tree with n = 2^d − 1 nodes and distinct real labels x_v. A node is a **local minimum** if its label is smaller than the labels of all its neighbours (parent and children). A label is read by probing the node. Find a local minimum with O(log n) probes.

Idea: Start at the root and repeatedly move to the child with the smaller label whenever the current node is not smaller than both children.

Algorithm in words

1. Probe the root v. It has no parent, so the invariant holds trivially.
2. If v is a leaf, stop: v is smaller than its parent (invariant), and it has no children, so v is a local minimum.
3. Otherwise probe both children. If x_v is smaller than both, stop: v is smaller than its parent and both children, so it is a local minimum.
4. Otherwise at least one child is smaller than v. Move to the **smaller** child w. Now x_w < x_v, so w is smaller than its parent v and the invariant holds. Go back to step 2.

Algorithms (CS302)
Project – Part 1
Divide and Conquer

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

Pseudocode

```
ALGORITHM LocalMin(T)
 v <- root;  probe x_v
 loop
   if v is a leaf then return v
   probe the children l and r of v
   if x_v < x_l and x_v < x_r then return v
   v <- the child with the smaller label
```

Recurrence relation

Each step down the tree costs 2 probes (the two children) and the root costs 1. If $P(d)$ is the number of probes for a tree with d levels, then

$$P(d) = P(d-1) + 2, \qquad P(1) = 1$$

Solution

Solving by repeated substitution:

$$P(d) = P(d-1) + 2$$
$$= [P(d-2) + 2] + 2 = P(d-2) + 4$$
$$= [P(d-3) + 2] + 4 = P(d-3) + 6$$

After k steps the pattern is

$$P(d) = P(d-k) + 2k$$

The recursion stops at the root level, when $d - k = 1$:

$$d - k = 1 \quad \Rightarrow \quad k = d - 1$$

Substituting back in:

$$\boldsymbol{P(d) = P(1) + 2(d-1) = 1 + 2d - 2 = 2d - 1}$$

Since $n = 2^d - 1$ we have $d = \log_2(n+1)$, so the number of probes is

$$\boldsymbol{2\log_2(n+1) - 1 = O(\log n).}$$

Example

(n = 15, labels in level order): 8, 4, 12, 2, 6, 10, 14, 1, 3, 5, 7, 9, 11, 13, 15.

Probe root = 8. Probe children: 4 and 12. 8 is not smaller than both -> go to 4
Probe children of 4: 2 and 6. 4 is not smaller than both -> go to 2

GROUP MEMBERS
Sana Munir Alam [24K-0573]
Adeena Asif [24K-0628]
Sharjeel Memon [24K-0555]

Program / Section: BCS-5E

Probe children of 2: 1 and 3. 2 is not smaller than both -> go to 1
1 is a leaf, smaller than its parent 2 -> local minimum. Probes = 1 + 2 + 2 + 2 = 7

This agrees with the bound $2d - 1 = 7$ for $d = 4$.

Code

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> x;
int n, probes = 0;
int probe(int v) { probes++; return x[v]; }

int main() {
    cout << "Enter n = 2^d - 1 (n >= 15): "; cin >> n;
    if (n < 15 || ((n + 1) & n) != 0) { cout << "n must be 2^d - 1 and >= 15\n"; return 1; }
    x.assign(n + 1, 0);
    cout << "Enter " << n << " distinct labels (level order): ";
    for (int i = 1; i <= n; i++) cin >> x[i];
    int v = 1, xv = probe(1);
    while (true) {
        if (2 * v > n) break;
        int xl = probe(2 * v), xr = probe(2 * v + 1);
        if (xv < xl && xv < xr) break;
        if (xl < xr) { v = 2 * v; xv = xl; }
        else { v = 2 * v + 1; xv = xr; }
    }
    cout << "Local minimum: node " << v << " with label " << xv << ", probes = " << probes << "\n";
    return 0;
}
```

Output

